

Name:

Class:

Date:

AP CSP Final Review Guide

Everything on one stack of pages



How to use this guide

Each Big Idea page lists what you must know. Check off each line you could explain to a friend; star anything you'd miss. Then use the error-analysis page after your practice exam.

The AP exam

- **Multiple choice:** 70 questions in 120 minutes (single-answer and “select two”).
- **Written response:** questions about YOUR Create Task program, answered with your Personalized Project Reference.
- The **Exam Reference Sheet** (AP pseudocode) is provided.
- Create Task $\approx$ **30%** of your score; multiple choice $\approx$ **70%**.

Big Ideas

1. Creative Development
2. Data
3. Algorithms & Programming (*the biggest part of the exam*)
4. Computer Systems & Networks
5. Impact of Computing

Test-day strategies

Read the LAST line first. Know what's being asked before you trace any code.

Eliminate first. Cross out answers that are definitely wrong; two choices are often clearly out.

Plug in test values. For 'which code works?' questions, try a boundary input like 0, 1, an equal pair, or an empty list.

Pace yourself. 70 questions in 120 minutes $\approx$ 1 minute 40 seconds each. Skip, mark, and come back.

Trace on paper. Make a quick table of variables. Never trace in your head.

Select-two questions. You must pick exactly two, and both must be correct to earn the point.

Watch absolute words. Answers with 'always', 'never', 'guarantees', or 'all' are often wrong in Big Idea 5.

Never leave blanks. There's no penalty for guessing.

Name: _____

Class: _____

Date: _____

Big Idea 1 · Creative Development

Topics 1.1–1.4

Collaboration (1.1)

- Diverse teams bring different perspectives → better solutions and less bias.
- Pair programming: driver types, navigator reviews; switch often.
- A computing innovation includes a program as an essential part (physical, software, or a concept like e-commerce).

Function & purpose (1.2)

- Purpose = WHY (problem solved / interest pursued). Function = WHAT it does.
- Input: data sent to a program (user, sensor, file, another program). Output: visual, audio, text, tactile.
- Event-driven programs run code when an event (click, key, timer) happens.

Design & development (1.3)

- Investigate (surveys, interviews, observation, user testing) → design → prototype → test.
- Iterative = revise using feedback. Incremental = build and test one piece at a time.
- Document throughout; comments don't run; credit any code you didn't write.

Errors (1.4)

- Syntax (breaks language rules) · Run-time (crashes while running) · Logic (wrong output) · Overflow (number too big).
- Find bugs with test cases, hand tracing, DISPLAY statements, debuggers.
- Test boundary values: at and just beyond the edges of valid input.

My weak spots

What I still need to review:

Name: _____

Class: _____

Date: _____

Big Idea 2 · Data

Topics 2.1–2.4

Binary (2.1)

- n bits $\rightarrow 2^n$ values; largest value $2^n - 1$; each extra bit doubles the values.
- Place values: 128 64 32 16 8 4 2 1.
- Same bits can mean different things (number, letter, color) depending on context.
- Overflow: too big for the bits. Roundoff: real numbers are approximated.

Compression (2.2)

- Lossless: exact original restored (text, code, medical images).
- Lossy: smaller, approximate (video, music, photos).
- How much a file shrinks depends on redundancy and the algorithm.

Information from data (2.3)

- Metadata = data about data; changing it doesn't change the data.
- Correlation $\neq$ causation. Look for a third factor.
- Cleaning makes data uniform. More data from a biased source is still biased.
- Big data may need parallel/distributed processing (scalability).

Programs with data (2.4)

- Filter, sort, transform, combine, visualize.
- Line = change over time · Bar = compare categories · Scatter = relationship.
- Read the columns: a question can only be answered if the data contains what it needs.

My weak spots

What I still need to review:

Date:

Topics 3.1–3.9

What I still need to review:

Name:

Class:

Date:

Big Idea 3 · Algorithms & Programming (part 2)

Topics 3.10–3.18

Lists & searching (3.10–3.11)

- APPEND (end), INSERT (shift right), REMOVE (shift left), LENGTH.
- Removing inside a loop: don't advance the index after a removal.
- Linear search: any list, up to n checks. Binary search: SORTED list, about $\log_2 n$ checks.

Procedures & libraries (3.12–3.14)

- Parameters are in the definition; arguments are in the call.
- RETURN sends a value back and ends the procedure.
- Procedural abstraction: use it knowing what it does, not how.
- Libraries/APIs: reuse tested code; read the documentation.

Random & simulations (3.15–3.16)

- RANDOM(a, b) includes a and b ; $(b - a + 1)$ values.
- Separate RANDOM calls are independent — probabilities multiply.
- Simulations simplify reality; what's left out can bias results.

Efficiency & limits (3.17–3.18)

- Reasonable: polynomial (n, n^2, n^3). Unreasonable: exponential (2^n), factorial ($n!$).
- Heuristic: good-enough answer when the best one takes too long.
- Undecidable: no algorithm solves all inputs (Halting Problem). Slow $\neq$ impossible.

My weak spots

What I still need to review:

Date:

What I still need to review:

Date:

Topics 5.1–5.6

Practice Exam Error Analysis

Learn from every missed question

Missed questions

Q #	Topic	Why I missed it*	What I should have done

*C = careless · V = vocabulary · T = tracing mistake · K = didn't know the concept · R = ran out of time

Tally by Big Idea

Big Idea	Missed
1 Creative Development	
2 Data	
3 Algorithms & Programming	
4 Systems & Networks	
5 Impact	

My plan

The two topics I'll review first, and how: